

Product Requirements Document (PRD) Daily Quest

1.1. *Daily Quest*

Dokumen ini menjelaskan kebutuhan produk Daily Quest sebagai aplikasi pribadi untuk mencatat dan memantau aktivitas harian. Produk ini ditujukan untuk membantu pengguna mengganti kebiasaan pencatatan manual yang selama ini hanya mengandalkan ingatan atau hafalan. Dengan sistem ini, pengguna dapat membuat daftar *task* harian, menandai pekerjaan yang telah selesai, melihat riwayat pencapaian, dan keluar dari akun dengan aman.

1.1.1. *Overview*

Daily Quest adalah aplikasi *web* pribadi untuk melakukan *tracking* terhadap aktivitas harian pengguna. Masalah utama yang ingin diselesaikan adalah pencatatan aktivitas harian yang masih dilakukan secara manual, sehingga pengguna mudah lupa terhadap kegiatan yang perlu dikerjakan. Aplikasi ini menyediakan tempat terpusat untuk menulis, memantau, dan mengevaluasi *task* harian secara lebih rapi.

Tujuan utama aplikasi ini adalah membantu pengguna membentuk kebiasaan harian yang lebih konsisten. Setiap *task* dapat dibuat oleh pengguna dan status penyelesaiannya akan kembali kosong pada hari berikutnya. Meskipun status harian mengalami *reset*, riwayat penyelesaian tetap tersimpan agar pengguna bisa melihat aktivitas yang sudah dilakukan pada hari-hari sebelumnya.

Aplikasi ini akan digunakan secara *online* dan di-*hosting* menggunakan Vercel agar dapat diakses dari perangkat *desktop* maupun *mobile*. Cakupan versi awal berfokus pada kebutuhan pribadi, sehingga tidak terdapat pembagian peran pengguna. Arah pengembangan produk dibuat sederhana, stabil, dan cukup fleksibel untuk ditambah fitur lanjutan apabila dibutuhkan di masa depan.

1.1.2. *Requirements*

Kebutuhan produk Daily Quest dibagi menjadi kebutuhan fungsional dan nonfungsional. Kebutuhan fungsional menjelaskan tindakan yang harus bisa

dilakukan oleh pengguna ketika memakai aplikasi. Kebutuhan nonfungsional menjelaskan kualitas sistem, seperti kemudahan penggunaan, keamanan akun, responsivitas tampilan, dan ketersediaan sistem saat diakses secara *online*.

Kategori	Kebutuhan	Prioritas
Fungsional	Pengguna dapat melakukan login ke aplikasi menggunakan akun pribadi.	Tinggi
Fungsional	Pengguna dapat melakukan logout agar sesi penggunaan dapat diakhiri dengan aman.	Tinggi
Fungsional	Pengguna dapat menambahkan task harian yang akan muncul pada dashboard.	Tinggi
Fungsional	Pengguna dapat mencentang task yang sudah selesai pada hari berjalan.	Tinggi
Fungsional	Status penyelesaian task harus otomatis kosong kembali ketika berganti hari.	Tinggi
Fungsional	Pengguna dapat melihat histori task dan status penyelesaian pada hari sebelumnya.	Tinggi
Nonfungsional	Tampilan harus responsif untuk desktop dan mobile.	Tinggi
Nonfungsional	Data akun dan histori task harus tersimpan secara online dan aman.	Tinggi
Nonfungsional	Antarmuka harus sederhana karena aplikasi digunakan untuk kebutuhan pribadi sehari-hari.	Sedang

Tabel 1 Daftar requirements Daily Quest.

Batasan penting pada versi awal adalah aplikasi tidak memerlukan sistem peran seperti admin, dosen, atau pengguna umum. Seluruh fitur hanya ditujukan

untuk satu pengguna yang memakai aplikasi sebagai alat pencatatan pribadi. Oleh karena itu, alur dan struktur data dibuat ringkas agar pengembangan lebih cepat dan fokus pada fungsi inti.

1.1.3. Core Features

Fitur inti Daily Quest disusun berdasarkan kebutuhan utama pengguna, yaitu mencatat *task* harian, menyelesaikan *task*, dan melihat histori. Fitur dibuat dengan pendekatan minimal namun tetap lengkap untuk kebutuhan awal. Setiap fitur harus dapat digunakan tanpa proses yang rumit agar aplikasi benar-benar membantu rutinitas harian pengguna.

Fitur	Deskripsi	Kriteria Berhasil
Login	Pengguna masuk ke aplikasi menggunakan akun pribadi.	Pengguna berhasil diarahkan ke dashboard setelah kredensial valid.
Logout	Pengguna keluar dari aplikasi dan sesi aktif dihentikan.	Pengguna kembali ke halaman login setelah menekan tombol logout.
Dashboard	Halaman utama yang menampilkan task hari ini dan status penyelesaiannya.	Task hari ini tampil rapi dan dapat langsung dicentang.
Tambah Task	Pengguna dapat menambahkan task harian baru.	Task yang ditambahkan langsung muncul pada daftar hari ini.
Centang Task	Pengguna dapat menandai task sebagai selesai untuk tanggal berjalan.	Status selesai tersimpan dan muncul pada histori hari tersebut.
Reset Harian	Status task kembali belum selesai pada hari berikutnya tanpa menghapus data histori.	Saat tanggal berubah, checkbox task kembali kosong dan riwayat hari sebelumnya tetap ada.
Histori Task	Pengguna dapat melihat catatan task yang selesai dan belum selesai pada hari sebelumnya.	Histori dapat ditelusuri berdasarkan tanggal atau urutan waktu terbaru.

Tabel 2 Daftar core features Daily Quest.

Konsep *reset* harian tidak berarti data *task* dihapus dari sistem. Aplikasi hanya mengosongkan status penyelesaian untuk tanggal baru, sementara catatan

penyelesaian pada tanggal sebelumnya tetap disimpan. Dengan pendekatan ini, pengguna tetap dapat mempertahankan daftar kebiasaan harian sekaligus melihat performa historisnya.

1.1.4. User Flow

Alur utama penggunaan Daily Quest dimulai ketika pengguna membuka halaman *login*. Setelah memasukkan kredensial yang valid, pengguna diarahkan ke halaman utama atau *dashboard*. Pada halaman ini, pengguna dapat melihat daftar *task* untuk hari berjalan, menambahkan *task* baru, dan mencentang *task* yang sudah selesai.

Ketika pengguna mencentang *task*, sistem menyimpan status penyelesaian berdasarkan tanggal hari itu. Jika pengguna membuka aplikasi pada hari berikutnya, daftar *task* tetap tersedia, tetapi status centang kembali kosong. Riwayat hari sebelumnya dapat dilihat melalui halaman histori sehingga pengguna tidak kehilangan jejak aktivitas yang sudah dilakukan.

No.	Langkah	Penjelasan
1	Buka aplikasi	Pengguna mengakses aplikasi Daily Quest melalui browser.
2	Login	Pengguna masuk menggunakan akun pribadi.
3	Lihat dashboard	Sistem menampilkan task untuk hari berjalan.
4	Tambah task	Pengguna menambahkan task harian sesuai kebutuhan.
5	Centang task	Pengguna menandai task yang sudah selesai.
6	Simpan status	Sistem menyimpan status penyelesaian berdasarkan tanggal.
7	Lihat histori	Pengguna membuka histori untuk mengecek catatan hari sebelumnya.
8	Logout	Pengguna keluar dari aplikasi setelah selesai menggunakan sistem.

Tabel 3 User flow utama Daily Quest.

Alur alternatif perlu disiapkan untuk kondisi ketika pengguna memasukkan kredensial yang salah atau menambahkan *task* kosong. Pada kondisi kredensial salah, sistem harus menampilkan pesan kesalahan dan tetap berada di halaman *login*. Pada kondisi judul *task* kosong, sistem harus menolak penyimpanan dan meminta pengguna mengisi nama aktivitas terlebih dahulu.

1.1.5. Architecture

Daily Quest direkomendasikan menggunakan arsitektur *client-server* sederhana. Pengguna mengakses aplikasi melalui *browser* pada perangkat *desktop* atau *mobile*. Tampilan aplikasi dijalankan melalui Next.js di Vercel, sedangkan autentikasi dan penyimpanan data dijalankan menggunakan Supabase.

Pada arsitektur ini, halaman aplikasi bertugas menampilkan antarmuka dan menerima input pengguna. Permintaan data dikirim ke layanan *backend* melalui fungsi *server-side* atau API internal. Layanan tersebut kemudian berkomunikasi dengan *database* untuk membaca daftar *task*, menyimpan status penyelesaian, dan mengambil histori berdasarkan akun pengguna.

Lapisan	Teknologi	Tanggung Jawab
Client	Browser pada desktop dan mobile	Menampilkan halaman login, dashboard, dan histori.
Frontend	Next.js dan Tailwind CSS	Mengatur tampilan, komponen, navigasi, dan interaksi pengguna.
Backend	Next.js Server Actions atau API Routes	Memproses permintaan tambah task, update status, histori, dan logout.
Authentication	Supabase Auth	Mengelola login, sesi pengguna, dan keamanan akses data.
Database	Supabase PostgreSQL	Menyimpan data akun, task harian, dan histori penyelesaian.

Deployment	Vercel	Menjalankan aplikasi secara online agar dapat diakses melalui internet.
------------	--------	---

Tabel 4 Arsitektur sistem Daily Quest.

Keamanan data perlu dijaga dengan memastikan setiap data *task* hanya dapat dibaca oleh pemilik akun. Sistem juga harus memvalidasi input sebelum menyimpan data ke *database*. Karena aplikasi digunakan secara pribadi, arsitektur tidak perlu dibuat kompleks, tetapi tetap harus cukup aman untuk digunakan secara *online*.

1.1.6. Database Schema

Skema *database* Daily Quest dirancang untuk memisahkan data akun, daftar *task*, dan status penyelesaian harian. Pemisahan ini penting karena *task* dapat digunakan berulang setiap hari, sedangkan status selesai atau belum selesai bergantung pada tanggal tertentu. Dengan model ini, sistem dapat melakukan *reset* harian tanpa menghapus riwayat.

Nama Tabel	Kolom Utama	Kegunaan
users	id, email, name, created_at	Menyimpan identitas akun pengguna. Jika Supabase Auth digunakan, tabel ini dapat berupa profile tambahan dari akun autentikasi.
tasks	id, user_id, title, description, is_active, created_at, updated_at	Menyimpan daftar task harian yang dibuat oleh pengguna dan dapat digunakan kembali setiap hari.
task_completions	id, task_id, user_id, completion_date, is_completed, completed_at	Menyimpan status penyelesaian task berdasarkan tanggal tertentu untuk kebutuhan histori.

Tabel 5 Skema database Daily Quest.

Relasi utama pada skema ini adalah satu pengguna dapat memiliki banyak *task*, dan satu *task* dapat memiliki banyak catatan penyelesaian harian. Kolom

completion_date menjadi kunci untuk membedakan status penyelesaian setiap hari. Jika tidak ada data penyelesaian untuk tanggal berjalan, sistem dapat menampilkan *task* sebagai belum selesai.

Pendekatan ini lebih aman dibandingkan menghapus atau membuat ulang daftar *task* setiap hari. Data histori tetap tersimpan, sedangkan tampilan harian tetap terasa baru karena status centang dihitung berdasarkan tanggal berjalan. Apabila fitur lanjutan dibutuhkan, struktur ini masih dapat dikembangkan untuk menambahkan kategori, prioritas, catatan, atau statistik mingguan.

1.1.7. *Tech Stack*

Berdasarkan kebutuhan aplikasi yang sederhana, pribadi, dan akan di-*hosting* di Vercel, teknologi yang paling sesuai adalah teknologi *web* modern yang ringan dan mudah dikembangkan. Rekomendasi utama adalah menggunakan Next.js dengan TypeScript agar tampilan, logika aplikasi, dan proses pengambilan data dapat dikelola dalam satu proyek. Untuk penyimpanan data dan autentikasi, Supabase menjadi pilihan praktis karena menyediakan *database* PostgreSQL dan sistem akun siap pakai.

Kategori	Teknologi	Alasan Penggunaan
Full-stack Framework	Next.js	Cocok untuk aplikasi yang akan di- <i>hosting</i> di Vercel dan mendukung rendering modern.
Bahasa Pemrograman	TypeScript	Membantu menjaga struktur kode lebih aman dan mudah dikembangkan.
Styling UI	Tailwind CSS	Mempercepat pembuatan tampilan responsive untuk desktop dan mobile.
Komponen UI	shadcn/ui	Menyediakan komponen siap pakai seperti form, button, card, dan dialog.
Autentikasi	Supabase Auth	Menyediakan login, logout, dan pengelolaan sesi pengguna.

Database	Supabase PostgreSQL	Menyimpan akun, task, dan histori secara online.
Validasi Data	Zod	Memvalidasi input seperti email, password, dan judul task.
Tanggal dan Waktu	date-fns	Membantu perhitungan tanggal untuk reset harian dan histori.
Deployment	Vercel	Menjalankan aplikasi secara online dan sesuai dengan rencana hosting.

Tabel 6 Rekomendasi tech stack Daily Quest.

Dengan susunan teknologi ini, Daily Quest dapat dikembangkan sebagai aplikasi kecil yang tetap rapi dan mudah dirawat. Next.js dan Vercel cocok digunakan bersama karena proses *deployment* dapat dilakukan langsung dari repositori proyek. Supabase membantu mempercepat pengembangan karena fitur akun dan *database* tidak perlu dibangun dari nol.

Untuk versi awal, aplikasi tidak perlu menggunakan arsitektur yang terlalu besar seperti *microservices* atau sistem peran kompleks. Fokus pengembangan sebaiknya diarahkan pada alur *login*, pencatatan *task*, *reset* harian, histori, dan pengalaman penggunaan yang nyaman. Setelah fitur inti stabil, pengembangan dapat dilanjutkan ke statistik kebiasaan, kategori *task*, atau pengingat harian.